# ANALYZING THE LIMITS OF RL-DRIVEN OPTIMIZATION ON CHARGAX

**Nael Belhaj Haddou**
s4431278

**Stan Ruijters**
s3208761

## ABSTRACT

Efficient control of electric vehicle charging stations requires balancing charging levels with fluctuating energy prices and differing user demands to maximize profit. The Chargax environment enables highly efficient simulation and RL-driven optimization of these scenarios. However, the potential of reinforcement learning may be constrained by the flexibility of the environment. Here, the limits of RL-driven optimization in Chargax are investigated. First, episode-specific upper bounds are estimated for various scenarios by overfitting agents on seeded episodes, investigating whether non-seeded agents can solve the environment. Then, the policy of both PPO and MBPO agents are studied by iteratively pruning unimportant features from the state vector, examining the difference in strategy between model-free and model-based RL through the features that each agent considers most important. Results show that only a small fraction of the available features is required for a PPO agent to reach the upper bound of a given scenario, whereas MBPO falls slightly behind due to unnecessary complexity. This demonstrates that little room for performance gains exist in Chargax, and further complexity needs to be added to properly study the potential of future RL developments.

## 1 INTRODUCTION

With the recent widespread adoption of electric vehicles, effective operation and control of charging infrastructure has become an increasingly complex problem. To maximize profit, an EV charging station must balance charge scheduling decisions with the cost of power, while taking into account the uncertainty of various user demands and changing grid prices across a full day. In recent years, reinforcement learning has proven to be a promising candidate for enabling high-performing autonomous control of these systems. Notable for this work is Chargax, a JAX-accelerated environment for training RL agents on EV charging station simulations. Previous work has shown that agents trained on Chargax are able to outperform agents following simple heuristics under various price and demand scenarios (Ponse et al., 2025).

Despite this, the extent to which RL agents can continue to improve on this environment is unclear. Compared to agents with trivial strategies, a Proximal Policy Optimization (PPO) agent does not improve significantly over them in certain charging scenarios (Ponse et al., 2025). Furthermore, our own empirical evaluations of other RL agents on Chargax have also not been able to outperform PPO, often only reaching similar performance levels.

These observations raise questions about the limits of the application of RL agents on the Chargax environment. Specifically, it remains unclear whether performance is constrained by limitations of current RL agents or the flexibility and simplicity of the environment. This work investigates this question through a set of experiments aimed to provide insights in the performance limits of both RL agents and the Chargax environment. First, a method is proposed to estimate the upper bound of various Chargax scenarios by forcing an RL agent to overfit on seeded episodes, allowing it to model and exploit future knowledge. Next, the redundancy of the state vector is examined by investigating the minimum number of required features an agent needs to perform near-optimally. Finally, this is repeated for a Model-Based Policy Optimization agent, examining strategies and advantages of model-based agents over model-free agents in partially observable scenarios.

## 2 Related Work

Besides the Chargax environment, various other studies have investigated the performance of reinforcement learning agents on other electric vehicle charging simulators. Early work in this field includes (Rigas et al., 2018), who proposed the EVLibSim environment in 2018. However, this environment can be optimally solved through trivial strategies, limiting potential advantages of RL agents. Another early environment includes Chargym (Karatzinis et al., 2022), implementing more realistic and complex state dynamics. Additionally, the authors trained several RL agents on it as well, showing that RL is able to outperform a heuristic agent which the authors themselves call far from optimal. Finally, Sustaingym Yeh et al. (2023) implements five different sustainable energy problems, including an EV charging environment. However, the authors conclude that both single-agent and multi-agent approaches perform no better than random actions, and a greedy heuristic achieves higher profit.

Other work has investigated the advantages of more complex RL algorithms on electric vehicle charging. Poddubnyy et al. investigated the performance of various other RL agents, including Dyna-Q, Dyna-Q+ and SARSA (Poddubnyy et al., 2023). However, the improvement over a naive rule-based algorithm was marginal. Most recently, in 2025, Jiang et al. showed Double Hypernetwork QMIX (Jiang et al., 2025), a multi-agent RL approach to maximizing profit in EV charging using coordinated double Q-learning agents and a mixing network to centralize training. This approach converges close to the theoretical upper bound of their environment, calculated using perfect knowledge of the randomness of the episode. Despite achieving state-of-the-art results on EV charging environments, a vanilla DQN agent is able to achieve approximately 2% less monthly profit than Double Hypernetwork QMIX relative to the upper bound, showing how traditional single-agent approaches remain potential solutions for near-optimal control of EV charging stations.

## 3 Method

### 3.1 Environment

The Chargax environment (Ponse et al., 2025) is a reinforcement learning framework for training and evaluating agents on simulations of EV charging scenarios. Compared to other EV charging environments (Karatzinis et al., 2022; Yeh et al., 2023), Chargax achieves a speedup of 100x-1000x through the use of JAX acceleration (Bradbury et al., 2018). This allows for significantly lower training times, enabling faster experimentation and more rapid algorithm design. An EV charging station in Chargax consists of a number of AC- and DC chargers, each connected through splitters to a main station battery that is linked to the grid. Naturally, the station is limited to charging as many cars as there are chargers available. The layout is static and does not change during training.

The state space can be split into an endogeneous and exogeneous part. The endogeneous state space contains state elements that are dependent on the actions of the agent in the environment. These elements include the battery level of each individual charging port and the station, as well as information on the cars that are currently charging. The exogeneous state space consists of factors that are unaffected by the decisions of the agent and change without external influence. This space includes information on the user and car profiles, information on the arrival rate of new cars, and the buy and sell prices of the electricity grid each timestep. The profiles in the exogeneous state space can be set by the user, changing how the scenario plays out.

The agent takes actions in the environment by assigning a change in power level to each individual charger. The transition function applies these changes to the chargers and updates the state of the connected cars according to the new charging levels. Finally, cars depart based on their battery level or time, and new cars arrive at the charging station. The episode reward is calculated as the total profit $\Pi(t)$ made from buying and selling energy to the grid at a given timestep $t$. Given the difference in net energy from the grid $\Delta E_{grid,net}(t)$, buy and sell prices $p_{buy}(t)$ and $p_{sell}(t)$, amount of energy used to charge all cars $\Delta E_{net}(t)$ and maintenance costs $c_{\Delta t}$, the profit can be calculated using Equation 1.

$$\Pi(t) = \begin{cases} p_{\text{sell}}(t)\,\Delta E_{\text{net}}(t) - p_{\text{buy}}(t)\,\Delta E_{\text{grid,net}} - c\,\Delta t, & \Delta E_{\text{grid,net}} > 0, \\ p_{\text{sell}}(t)\,\Delta E_{\text{net}}(t) - p_{\text{sell,grid}}(t)\,\Delta E_{\text{grid,net}} - c\,\Delta t, & \Delta E_{\text{grid,net}} \leq 0. \end{cases} \tag{1}$$

The goal of the reinforcement learning agent is to maximize the profit across a full episode, corresponding to a full day, by managing the power delivered to cars based on both user demand and fluctuating electricity prices. It is important to note that the environment also takes user preferences into account, by penalizing the reward based on several metrics of user satisfaction $c(t)$ at a given timestep $t$, factored by a constant $\alpha_c$:

$$r(t) = \Pi(t) - \sum_c \alpha_c c(t) \tag{2}$$

Consequently, the agent can be trained to prioritize specific points of user satisfaction. Examples of $c(t)$ include the number of rejected customers, the time taken to charge a car and the charge level that a car leaves at. Since the experiments in this work do not focus on this functionality, we set $\alpha_c = 0$ for all $c(t)$, effectively allowing the agent to fully maximize profit with no regard for user satisfaction. Thus, $r(t) = \Pi(t)$ for all experiments.

### 3.2 Upper bound

The reward upper bound of any given episode is nontrivial to find, since determining the upper bound is a separate optimization problem. Here, we only consider the unconstrained profit upper bound. Nevertheless, the method used is applicable with constraints as well.

To find the profit upper bound in a given episode, a PPO agent is continuously trained on seeded episodes that each represent the same day with the same randomization. The resulting overfitting is not an issue, since the main goal is to exploit the patterns in the episode as much as possible to generate profit. Since the upper bound will vary on each day depending on car arrivals, this experiment is repeated on multiple days in the same scenario. This gives an estimation of the profit upper bound in an average episode at those traffic and scenario settings. Note that this definition of the upper bound is not the definitive limit of the environment, since different randomization may allow for higher obtainable rewards. Nevertheless, it allows for an accurate estimate of the maximum performance that an RL agent is able to achieve during training.

### 3.3 State vector importance

The observation that the agent receives each timestep is extensive, containing information on various aspects of the environment. The main environment encodes temporal data and the station battery state, as well as buy and sell prices up to five timesteps in the future. Additionally, each individual charging port adds information on the current charging session, as well as constraints of the charging car. In total, the size of the state vector is $17n + 20$, with $n$ as the number of chargers. Due to the type of information each element encodes, some observations are more important to the agent than others. Consequently, the agent may not require all observations to find an optimal policy.

To investigate what types of observations contribute most to the strategy of the agent, an ablation study is performed. Due to the size of the state space, as well as the similarity of certain observations, ablating single elements is likely to only degrade performance marginally. Thus, we group the observations into one of six categories, each pertaining to a unique aspect of the agent's perception of the environment. The categories are shown in Table 1. Agents are trained with one of the categories ablated from the state vector, measuring the effect on performance. Additionally, the agent is trained without a state vector, providing a lower bound for the performance of PPO.

To find the importance of individual state elements, we perform Recursive Feature Addition (RFA) (Hamed et al., 2017). RFA starts by training an agent on an empty set of observations. Then, additional agents are trained by iteratively adding individual state elements to the observation set. The element with the largest mean increase in performance is considered the most important one, and permanently added to the observation set. The process is then repeated until all features are added. Through RFA, a precise ranking of the importance of observations can be obtained, as well

**Algorithm 1** State vector feature ranking through RFE and RFA

Initialize $n$ features $f_1, \ldots, f_n$ , elimination batch size $k$
Initialize current feature set $F_{\text{current}} \leftarrow \{f_1, \ldots, f_n\}$
**while** $|F_{\text{current}}| > 1$ **do**
    Train agent using features $F_{\text{current}}$
    Extract trained actor input weights
    Remove $k$ lowest-weight features from $F_{\text{current}}$
**end while**
Initialize remaining feature set $F_{\text{remaining}} \leftarrow F_{\text{current}}$, ranked feature list $F_{\text{ranked}} \leftarrow [\,]$
**while** $F_{\text{remaining}} \neq \emptyset$ **do**
    **for** all features $f \in F_{\text{remaining}}$ **do**
        Train agent using features $F_{\text{ranked}} \cup \{f\}$
        Evaluate final obtained profit
    **end for**
    Select feature $f^*$ with highest final profit
    Add $f^*$ to $F_{\text{ranked}}$ and remove from $F_{\text{remaining}}$
**end while**
**return** $F_{\text{ranked}}$

as an estimate for the minimum number of observations the agent needs to achieve near-optimal performance on Chargax.

Given $k$ features, a full run of RFA requires $\frac{k(k+1)}{2}$ agents to be trained. Thus, performing RFA on the full Chargax state vector becomes increasingly more inefficient as the number of chargers rises. Additionally, due to the similarities between certain features, precisely calculating the importance of each feature is redundant, since similar features will likely have similar importance. To combat this, we first perform Recursive Feature Elimination (RFE) to filter out the most irrelevant features effectively (Priyatno & Widiyaningtyas, 2024). Each iteration, a single agent is trained on a set of features, starting with the full state vector. Since each feature is normalized, the corresponding input weights of the actor network provide a rough estimate of the relative importance the agent assigns to each feature. Given this ranking, multiple of the lowest-ranking features can be removed from the state vector simultaneously. By doing this, the least important elements can efficiently be filtered out of the state vector first, significantly reducing the training time required for full-state RFA.

### 3.4 MODEL-BASED RL

The final set of experiments is focused on investigating the differences in policies between model-free and model-based reinforcement learning. Next to the PPO agent provided by the Chargax environment, a Model-Based Policy Optimization (MBPO) agent is trained as well (Janner et al., 2021). An MBPO agent uses previously obtained transitions to learn the transition function using an ensemble of dynamics networks. Each step, the agent performs short rollouts of previously observed states to generate additional transitions. The combination of real and synthetic transitions is then used to update a policy optimization agent. The addition of short synthetic rollouts allows the agent to be optimized with respect to near-term consequences of actions, whilst limiting model bias caused by longer rollouts. Consequently, the agent can implicitly plan its actions on a short horizon, potentially allowing for changes to its policy.

Table 1: Types of observables encoded in the full Chargax state vector.

| Group | Category | Examples |
|---|---|---|
| 1 | Car state | Time until car leaves, battery level now/on arrival |
| 2 | Charger constraints | Maximum in/outtake (KWh), sensitivity |
| 3 | Charger capabilities | Charge rates, charge threshold, current charge level |
| 4 | Temporal context | Timestep, day of year, workday |
| 5 | Grid prices | Buy and sell prices between now and 5 future timesteps |
| 6 | Station battery | Battery level, battery percentage, maximum charge rate |

The abilities of model-based agents show promise for improving complex EV charging optimization problems Rahali et al. (2025). Through planning, model-based agents can effectively evaluate the long-term impact of charging decisions under uncertain electricity prices and user behavior. Additionally, this can also increase robustness of the policy, since the agent can maintain the required level of safety and sustainability present in real-world applications better than model-free approaches.

To investigate the differences between policies found by model-free and model-based RL, the combination of RFE and RFA is run for an MBPO agent as well. By doing this, the differences in pruned features provide insights in what each agent type finds important for optimizing policies. Furthermore, the planning abilities of MBPO may allow it to find policies that solve the environment with fewer required observations.

Traditionally, the policy optimization algorithm used for MBPO is SAC. Since the main aim of these experiments is to compare differences in strategies between model-free and model-based RL, and PPO is the model-free baseline used, a PPO agent is used for policy optimization instead. This limits the differences of the agents to the presence of the learned dynamics model, without including further differences between policy optimization methods. This change provides both advantages and disadvantages for MBPO. PPO can provide more stability under model error than SAC due to not relying on bootstrapped Q-values, causing less severe divergence when the dynamics model is inaccurate. Additionally, since SAC relies on long-horizon Q-target bootstrapping, PPO may be more fit for the short-horizon trajectory updates that MBPO relies on. However, SAC allows for higher sample efficiency and potentially higher performance when the dynamics model is stable due to its off-policy nature compared to PPO.

### 3.5 Experimental details

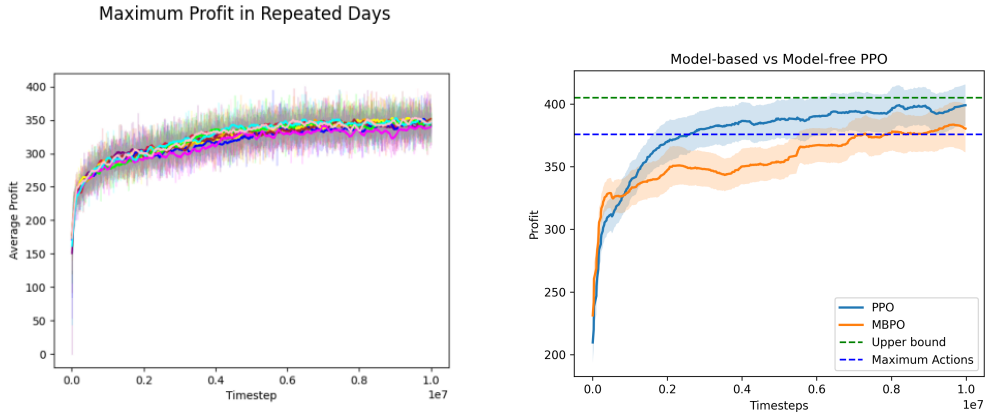
To ensure validity of the experiment code, a reproduction of the original results of the Chargax paper (Ponse et al., 2025) is conducted. Unless mentioned otherwise, all experiments are run on the shopping scenario, using medium traffic arrival. Results are averaged across 10 repetitions, and reported using a 95% confidence interval. An overview of all hyperparameters used to initialize the Chargax environment can be seen in Appendix B. To select elements to prune or add in RFE and RFA respectively, each iteration is run for 10 repetitions, and the average importance of each element is used for selection. RFE is performed until 20 features remain, then RFA finds a precise ranking among the remaining features.

## 4 Results

### 4.1 Upper bound

The upper bound was estimated with 10 agents, each trained on a different day repeating for all episodes. The experiment is rerun for 10 repetitions with the same days to account for training stochasticity. All days are set in the residential scenario with low traffic. As shown in Figure 1b, the profit reaches approximately 372 on an average day and in the average repetition. This is close to the profits of 375 reached on average by PPO in the same scenario. The largest profit on average in a repetition is about 393. It can also be noted that the deviation between repetitions is approximately 3.45, which is relatively marginal compared to average and maximum profit. This shows individual episodes of the same environment do not significantly differ from each other, which may strengthen the simplicity of the environment.

A performance comparison between PPO and MBPO is shown in Figure 1b, along with the estimated upper bound for this scenario. As can be seen, the addition of the dynamics model does not directly improve the profit that the agent can achieve. Since PPO is able to achieve a profit that is close to the upper bound of this scenario, additional planning abilities are not required for the agent to solve the environment. In contrast, due to the additional complexity and uncertainty added by synthetic rollouts, the MBPO agent takes more steps to converge than PPO.




(a) Upper bound seeded learning curves

(b) Mean learning curves of PPO and MBPO

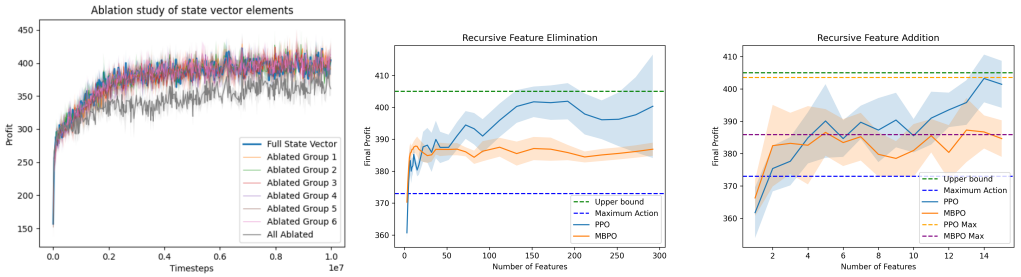
Figure 1: Profit for the average day at different lengths of training. Results for (a) were obtained on the residential scenario with low traffic. Results for (b) were obtained on the shopping scenario with medium traffic. Each curve represents the mean of a separate seed. The standard deviation is quite small with no repetition doing particularly well, suggesting the differences between seeds are marginal over a full episode. The average profit reached in all repetitions is approximately 370 and the maximum reached is about 393. MBPO shows a notable performance drop compared to PPO, likely due to unnecessary complexity in an environment that PPO is already able to solve.

### 4.2 Limiting information

The result of ablating the types of observations shown in Table 1 from the state vector is shown in Figure 2a. For all observation types, ablating them provides negligible differences in performance. From this, it can be reasoned that the agent is not dependent on any certain type of feature to reach its optimal performance. If a feature type is ablated that is more important to the policy, the remaining features are sufficient to construct a different policy that also solves the environment.

The learning curve of the agent without any observations is notably lower than those with only one group ablated, showing that the agent does require some observations to achieve similar performance to an agent with full knowledge. This curve approaches the approximate result that is obtained when the agent chooses to charge maximally each timestep, suggesting that a fully blind agent converges to a similar policy. It does not consistently reach this baseline, since the continuous nature of the action space limits the agent from consistently selecting the highest amount of charge. Furthermore, it confirms that always charging maximally is one of the strongest baselines that does not take any state information into account, and is able to provide adequate results in EV charging.

The lack of performance drops shown in the ablation study is further supported by Figure 2b, which shows how iteratively pruning the least important features from the state vector affects both PPO and MBPO. The achieved profit remains optimal until approximately 50 features remain, after which performance notably drops. The final profit continuously lowers until the agent has no information, again converging to a slightly lower profit than is achieved with maximum actions. When the 15 most important features are ranked starting from an empty state vector, optimal performance can be achieved with even less information, as shown in Figure 2c. The PPO agent is able to improve over the heuristic with a single feature, and reaches approximately 400 profit with 13 features in total. Interestingly, the MBPO agent is able to outperform the PPO agent in environments with little information, achieving similar performance to an MBPO agent with full information within 5 features. By learning a dynamics model, the MBPO agent is able to infer the missing state information from temporal patterns better than a PPO agent, allowing it to perform accurate actions in scenarios with a significant lack of information.




(a) Ablation study (b) Recursive Feature Elimination (c) Recursive Feature Addition

Figure 2: Effects of removing features from the environment through (a) selective pruning, (b) recursive feature elimination and (c) recursive feature addition. Results were obtained on the shopping scenario with medium traffic. The upper bound was obtained through the method described in Section 3.2. Results are averaged across 10 repetitions and reported with a 95% confidence interval. Training hyperparameters are as shown in Appendix B. Removing all of a single type of feature (a) does not impact performance, as the agent is able to solve episodes through different strategies using the remaining information. As shown in (b) and (c) both PPO and MPBO are able to achieve their respective near-optimal performance with less than 15 features. MBPO can achieve its best performance faster than PPO, but generally struggles to approach the upper bound.

### 4.3 Feature importance

From the RFA study, an approximate ranking can be determined for which features an agent finds most important relative to an empty state vector, by looking at the order in which they are added. Similarly, the RFE study provides a feature ranking relative to the full state vector. The ranking for both PPO and MBPO agents is shown in Table 2. Note that only five features are ranked, while the RFA study was performed over 15 features. This is due to the fact that features related to the car state contain multiple of the same type of feature; one per charger in the environment. Since these provide essentially similar information, multiple instances of one feature are grouped together. The top-5 of both PPO and MBPO correspond to all 15 features present in the RFA study.

Labeling the shown features with their corresponding categories defined in Table 1, it can be seen that most of the important features provide information about the state of the connected cars. More specifically, this includes data on the remaining time before a car leaves or when its battery is fully charged. Out of all features, the car state features correlate closest with the reward signal, since profit is mostly determined by how much a car is charged. Thus, information on when a car is leaving and how much time remains until the desired battery level is reached allows for accurate modeling of the reward without requiring additional information. The closer the observation is to these goals, the better the reward can be modeled.

Table 2: Ranking of most important features for both PPO and MBPO, along with their categories as defined in Table 1. Results were obtained on the shopping scenario with medium traffic. Importance is defined as the order in which RFA adds features to the state vector, given performance averaged across 10 repetitions. Training hyperparameters are as shown in Appendix B. Both PPO and MBPO require information on car states the most, since these features correspond closest with the reward signal. PPO requires some information on temporal context as well, since it is unable to effectively model daily patterns otherwise.

| | **PPO** | | **MBPO** | |
|---|---|---|---|---|
| Rank | Feature | Category | Feature | Category |
| 1 | Training Timestep | Temporal | Time until car leaves | Car state |
| 2 | Time until car leaves | Car state | Total battery charged | Car state |
| 3 | Time until desired battery | Car state | Charge sensitivity | Car state |
| 4 | Current day of year | Temporal | Current charger output | Car state |
| 5 | Current car intake | Car state | Maximum AC charge rate | Car state |

Next to this, the PPO agent also requires some temporal structure given by the day of the year, which MBPO does not. The day of the year can capture information related to daily or seasonal patterns that influence the charging demand of customers, including arrival and departure rates and requested charge. The MBPO agent is able to infer these patterns better than a PPO agent due to its ability to sample future trajectories. Consequently, information on the time of day is less important for MBPO, and pruned early during RFE.

Interestingly, the PPO agent considers the current training timestep to be the most important feature. Adding this feature corresponds to approximately half the difference in profit between a normal agent and a blind one, despite not providing any information on the state itself. Several reasons can be attributed to this. Firstly, the training timestep can allow the agent to model phases of the day, since the number of timesteps each episode is similar. This allows the agent to model general daily patterns, such as on- and off-peak hours and price changes. Secondly, the future cumulative reward of an episode decreases approximately monotonically with time, since individual actions do not drastically change the state long-term. Thus, information on the timestep alone can allow the value function to accurately estimate the value of a state, which would provide a notable increase in performance. Finally, when having to add a single feature to an empty state vector, the timestep is the most unique option, since most features are dependent on other observations and cannot easily have their reward signal modeled due to lack of other important information about the state. Combined with the aforementioned reasons, this causes the timestep to give the most advantages when adding a single feature. The MBPO agent is less dependent on the timestep, since it can infer its state more easily through use of the dynamics model. Consequently, RFE prunes the timestep as one of the first features. Note that this reasoning cannot be generalized to all RL environments, as it assumes an environment in which episodes are of fixed length, where future reward decays smoothly over time and single actions do not drastically alter the state long-term.

## 5 Discussion

The low standard deviation in the average profit achieved in each repetition suggests that training randomness is not a big issue for PPO to find good solutions to any given day. Another notable result of the upper bound experiment is that the 375 average profit reached is extremely close to the average profit of PPO of 373 in the original environment with the same scenario. This means PPO likely solves the low traffic scenario of the environment, since stochasticity does not meaningfully worsen results. Thus, we conclude that PPO is able to optimally solve this scenario. Similarly, PPO approaches the upper bound for a shopping scenario with medium traffic as well. Nevertheless, more scearios need to be examined to conclusively determine that PPO is able to solve Chargax in all possible scenarios.

The MBPO agent shows a drop in performance compared to the PPO agent. While this is likely due to the added complexity in a situation where this is unnecessary, other model-based algorithms may reduce this gap. Additionally, while PPO was used as the underlying policy optimization algorithm, the original MBPO algorithm uses SAC. Additionally, other algorithms can be used as well. Varying the optimization algorithm used may positively impact the performance of the agent. However, adapting the algorithm does introduce more differences between the model-based and model-free agents besides the synthetic rollout, so changes in strategy may be harder to attribute to certain characteristics.

The main function of Chargax is to enable efficient RL benchmarking on real-world applications (Ponse et al., 2025). For real-world scenarios, most observation types present in the full Chargax state vector are features that most charging stations will have continuous access to. For instance, consider the date and time, pricing data and information on the station battery and charger capacity. These are types of features that can be obtained in all situations, and will only be unavailable when network connections or sensors fail to function. An exception to this may be the features corresponding to the car state and user demands, since not all chargers extract this information. However, as shown in the ablation study, removing features from this category still allows the agent to achieve optimal performance. Thus, from a practical perspective, these highly low-information states rarely occur, and deploying a PPO agent would be preferred over a model-based approach in almost all scenarios.

Finally, a key feature of Chargax is its ability to also model user satisfaction through various metrics, and penalizing the reward accordingly (as shown in Equation 1.) This work has omitted the use of these metrics, since they limit the maximum reward an agent can achieve. Despite this, including user satisfaction in the reward calculation makes the environment more complex, potentially lowering the reward PPO converges to and increasing the number of features required to achieve optimal performance. The closer the latter is to the total number of features in the environment, the more potential exists for improvements and strategies found by other RL algorithms.

## 6 Conclusion

This work has evaluated the limits of the application of reinforcement learning agents to the Chargax environment, which simulates profit gain of EV charging stations in various situations. This has been achieved through a method of estimating the upper bound of a given scenario, by forcing an agent to overfit on a seeded episode to optimize the reward as much as possible. The results show that a non-seeded PPO agent approximately performs equally on any given episode as a seeded agent, suggesting that traditional RL agents are able to find near-optimal policies given enough information about the state.

To investigate the minimal amount of information required for an agent to find an optimal policy, features were iteratively added to an empty state vector to determine what the agent deems most important. This demonstrated that only a fraction of the total number of features are required for an agent to perform optimally. In scenarios with limited information, a model-based agent is able to outperform PPO through. However, a lack of this much information is not common in real-world applications, so model-free agents will suffice in most cases.

Overall, these findings suggest that the Chargax environment presents a scenario in which traditional model-free agents are able to mostly reach all the performance that can theoretically be achieved. Future work may focus on increasing the complexity of the environment to allow more potential improvements, by incorporating strict user constraints or adding additional features that further complicate customer behavior and reward signals. By doing this, Chargax may be able to better differentiate strategies between RL algorithms, enable potential rewards beyond what PPO is able to achieve and continue driving research developments on the application of RL-driven optimization in real-world environments.

## References

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, et al. Jax: composable transformations of python+ numpy programs. 2018.

Tarfa Hamed, Stefan Kremer, and Rozita Dara. *Recursive Feature Addition: a Novel Feature Selection Technique, Including a Proof of Concept in Network Security ABSTRACT Recursive Feature Addition: a Novel Feature Selection Technique, Including a Proof of Concept in Network Security*. PhD thesis, 03 2017.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization, 2021. URL https://arxiv.org/abs/1906.08253.

Kun-Yan Jiang, Wei-Yu Chiu, and Yuan-Po Tsai. Profit maximization for electric vehicle charging stations using multiagent reinforcement learning. *Sustainable Energy, Grids and Networks*, 44:102009, 2025. ISSN 2352-4677. doi: https://doi.org/10.1016/j.segan.2025.102009. URL https://www.sciencedirect.com/science/article/pii/S2352467725003911.

Georgios Karatzinis, Christos Korkas, Michalis Terzopoulos, Christos Tsaknakis, Aliki Stefanopoulou, Iakovos Michailidis, and Elias Kosmatopoulos. *Chargym: An EV Charging Station Model for Controller Benchmarking*, pp. 241–252. 06 2022. ISBN 978-3-031-08340-2. doi: 10.1007/978-3-031-08341-9_20.

Andrey Poddubnyy, Phuong Nguyen, and Han Slootweg. Online ev charging controlled by reinforcement learning with experience replay. *Sustainable Energy, Grids and Networks*, 36:101162,

2023. ISSN 2352-4677. doi: https://doi.org/10.1016/j.segan.2023.101162. URL https://www.sciencedirect.com/science/article/pii/S2352467723001704.

Koen Ponse, Jan Kleuker, Aske Plaat, and Thomas Moerland. Chargax: A jax accelerated ev charging simulator. 07 2025. doi: 10.48550/arXiv.2507.01522.

Arif Mudi Priyatno and Triyanna Widiyaningtyas. A systematic literature review: Recursive feature elimination algorithms. 9:196–207, Feb. 2024. doi: 10.33480/jitk.v9i2.5015. URL https://ejournal.nusamandiri.ac.id/index.php/jitk/article/view/5015.

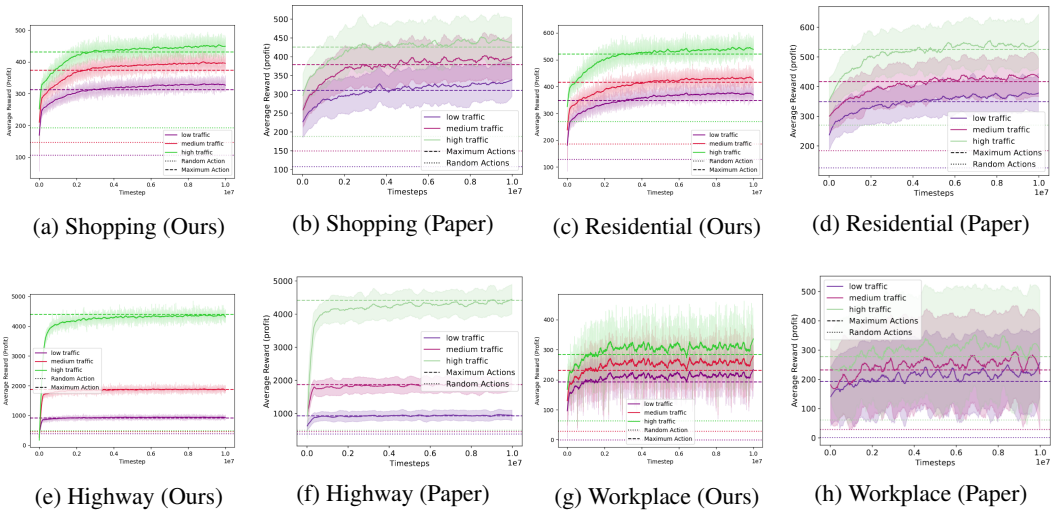
Hicham Rahali, Andrey Poddubnyy, and {Phuong H.} Nguyen. A comparison of model-based and model-free offline reinforcement learning methods for electric vehicle smart charging optimization. In *2025 IEEE Power and Energy Society General Meeting, PESGM 2025*, United States, November 2025. Institute of Electrical and Electronics Engineers. doi: 10.1109/PESGM52009.2025.11225472. 2025 IEEE Power & Energy Society General Meeting, PESGM 2025, PESGM 2025 ; Conference date: 27-07-2025 Through 31-07-2025.

Emmanouil S. Rigas, Sotiris Karapostolakis, Nick Bassiliades, and Sarvapali D. Ramchurn. Evlibsim: A tool for the simulation of electric vehicles' charging stations using the evlib library. *Simulation Modelling Practice and Theory*, 87:99–119, 2018. ISSN 1569-190X. doi: https://doi.org/10.1016/j.simpat.2018.06.007. URL https://www.sciencedirect.com/science/article/pii/S1569190X18300911.

Christopher Yeh, Victor Li, Rajeev Datta, Julio Arroyo, Nicolas Christianson, Chi Zhang, Yize Chen, Mehdi Hosseini, Azarang Golmohammadi, Yuanyuan Shi, Yisong Yue, and Adam Wierman. SustainGym: Reinforcement learning environments for sustainable energy systems. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, New Orleans, LA, USA, 12 2023. URL https://openreview.net/forum?id=vZ9tA3o3hr.

## A REPRODUCTIONS

Before performing other experiments, the results of the original Ponse et al., 2025 were reproduced. The parameters used were the same as in said paper and can be found in Appendix B. Additionally, a comparison of the results produced by our code and those included in the Chargax paper can be found in Figure 3.




(a) Shopping (Ours) (b) Shopping (Paper) (c) Residential (Ours) (d) Residential (Paper)

(e) Highway (Ours) (f) Highway (Paper) (g) Workplace (Ours) (h) Workplace (Paper)

Figure 3: A comparison of results from our reproduction and the original paper results (e-h). It can be seen that for each scenario, results closely match for all levels of traffic. This confirms that the results of our other experiments are run in the same setting as the original paper.

The results in Figure 3 match closely with the results of the original paper. This shows that the code used here is correct for chargax and PPO and that the original paper's results are reproducable.

## B HYPERPARAMETER SETTINGS

Table 3: Hyperparameters and Environment Parameters

| **Parameter** | **Value** |
|---|---|
| Total timesteps | $10^7$ |
| Learning rate ($\alpha$) | $2.5 \times 10^{-4}$ (annealed) |
| Discount factor ($\gamma$) | 0.99 |
| GAE $\lambda$ | 0.95 |
| Max grad norm | 100.0 |
| Clipping coefficient $\epsilon$ | 0.2 |
| Value function clip coefficient | 10.0 |
| Entropy coefficient | 0.01 |
| Value function coefficient | 0.25 |
| Vectorized environments | 12 |
| Rollout length (steps) | 300 |
| Number of minibatches | 4 |
| Update epochs | 4 |
| Minibatch size | 900 |
| Batch size | 3600 |
| Minutes per timestep $\Delta t$ | 5 |
| Discretization factor | 10 |
| Episode length | 24 hours |
| Number of Chargers | 16 |
| Number of DC Chargers | 10 |
| Sell price to customers ($p_{\text{sell}}$) | 0.75 |
| Reward coefficients $\alpha_c$ (Eq.2) | 0.0 |
| MBPO model rollout horizon | 5 |